CO1035: Game Concepts
Lab 1C – Conditionals and Player Input

1. Introduction

- The purpose of this lab is to demonstrate to you what conditionals and Player Input are.
- In the previous lab, we introduced the TL-Engine and used it to manipulate some simple 3D graphics. In this lab, you will learn about selection, using if-statements, and interacting with your program using mouse and keyboard input.
- Remember that it is useful for you to keep a copy of the code you write so that you can reference or reuse it in later programs.
- This is a very important lab. Make sure that you understand it by heart, as both concepts represent the foundation of all games.

2. The "if" statement (Read only)

a. Selection is the ability to execute code only if some condition is met (e.g., execute a block of code if the condition is true, or not execute a block of code if the condition is not true.

b. In C++, you can exercise this control through the use of the word "if". The general form of an if statement is:

```
if (condition)
{
// conditional execution
}
```

c. The condition is a programming statement which equates to either true or false. Any expression which equates to a number can also be used.

d. The following are examples of the use of the if statement:

```
if ( a == b ) //if the first Variable is equal to the second Variable
{
      // do something
}
if ( a != b ) //if the firstVariable is not equal to the secondVariable
{
      // do something
}
if ( a < b ) //if the firstVariable is less than the secondVariable
{
      // do something
}
if ( a > b ) //if the firstVariable is greater than the secondVariable
```

```
{
	// do something
}
if ( myEngine->KeyHit( [KeyCode] )) //if the key with the provided keycode is hit
{
	//do something
}
if (!myEngine->KeyHeld( [KeyCode] )) //if the key with the provided keycode is
NOT held (notice the use of !)
{
	//do something
}
if ( a + b < c) //if the sum of the firstVariable and secondVariable is less than
the thirdVariable
{
	// do something
}
if ( a * b > 100) //if the product of the firstVariable and secondVariable is
greater than 100
{
	// do something
}
```

e. Note that a conditional statement does not execute a command, but is used to make decisions about executing commands.

3. Checking for Player Input

a. The 3D engine monitors the keyboard and mouse input for its own window. We can ask the engine if particular keys or mouse buttons have been pressed; the methods to do this are:

```
bool KeyHit( EKeyCode ); // Returns true when a given key or button is first pressed down.

// Example usage: if (myEngine->KeyHit( Key_A )) { Do something }
bool KeyHeld( EKeyCode ); // Returns true as long as a given key or button is held down.

// Example usage: if (myEngine->KeyHeld( Mouse_LButton )) { Do something }
```

Note: The above code snippets show the method's signatures – the return type of the method (bool in both cases), the method name (e.g. KeyHit), and the parameter type (EKeyCode). You should not copy the method signature into your program (otherwise the compiler will think you are trying to redefine the methods!). When you call the methods, you can assign the result to a variable of the correct return type or, since the return type is bool, use the method call as the condition in an "if" statement, and you must pass in a parameter of the required type. The examples provided as comments show how the methods are used.

b. You must supply a special key code to indicate which key (or button) you want to test ("Key_A" and "Mouse_Lbutton" were used in the examples above). The full list of key codes can be found on Blackboard.
c. Notice that mouse buttons are considered to be "keys" also.
d. There is a slight difference between the two input methods provided above. The first ("KeyHit") returns true once, when the key is first pressed, then it returns false until you let go of the key and press it again.
e. The second method ("KeyHeld") will always return true as long as the key is held down. We will see how this difference is important soon.
f. Create a new TL-Engine project named **Lab1C_Controls_Project**, and open the project
g. Add code to create an FPS camera; you may want to position it at (0, 0, -50). You should be able to do this without looking at last week's lab sheet.

4. Keyboard Input

a. In your game loop, add code to check for when the "W" key is hit. Whenever the key is hit, if the y-position of the sphere is less than 30, then it should be increased by 5.
b. In your game loop, add code to check for when the "S" key is hit. Whenever the key is hit, if the y-position of the sphere is greater than 10.0, then it should be decreased by 5.
c. In your game loop, add code to check for when the "D" key is hit. Whenever the key is hit, the speed of the sphere should be increased by 10%.
d. 4. In your game loop, add code to check for when the "A" key is hit. Whenever the key is hit, if the speed of the sphere is greater than 0, then it should be decreased by 10%.
e. Now add and display a cube in your scene, then try using the 'if' statement by typing the code below into the game loop section:

```
if ( myEngine->KeyHeld( Mouse_RButton ))
{
	cube->MoveX( 0.05 );
}
else if ( myEngine->KeyHeld( Mouse_LButton ))
{
	cube->MoveX ( -0.05 );
}
```

This code tests if the right mouse button is held and moves the cube right; else if the left mouse button is held, it moves the cube left. Run your project to test this out.

f. Are you finding it difficult to keep the camera still while you move the cube with your mouse? Update your program to use a **'kManual'** camera instead of a **'kFPS'** by changing your camera creation code to the code below:

```
ICamera* myCamera;
```

```
myCamera = myEngine->CreateCamera(kManual);
myCamera->SetPosition(0, 0, -50);
```

This means the camera cannot be moved with the mouse anymore, so you will be able to easily move your cube with a click of the mouse buttons.

g. Change the code you added to move your cube to use the **'KeyHit'** method instead of 'KeyHeld'. Run your program - can you see the difference? Now change it back to **'KeyHeld'**.
h. Add another 'if' statement by typing the code below into the game loop section. With the latest version of TL-Engine, usually this block of code is automatically created in your while loop.

```
if ( myEngine->KeyHit( Key_Escape ))
{
	myEngine->Stop();
}
```

This code tests if the escape key has been pressed and stops the engine (and program) if it has. You may want to add this to all your projects, so you don't need to use 'Alt+F4'. Run your program and test quitting by pressing the Escape key.

5. Keyboard Control

a. Change the code you added in the last section for moving your cube to move the cube right when you hold the 'D' key (instead of the right mouse button) and move left when the key 'A' is held (instead of the left mouse button).
b. Now add code to move the cube forwards and backward (Z axis) with the keys 'W' and 'S'.
c. Finally, add code to move the cube up and down (Y axis) with 'Q' and 'E'.
d. You can now fully control the movement of a model. Now we will see what happens when we control the rotation.
e. Now change your camera type back to **kFPS**, and remove the code for setting its position to (0, 0, -50). Moving the camera can help in the tasks to follow.
f. Add a new "Grid.x" mesh and display a grid in your scene.
g. Change your code to load "Arrow.x" instead of "Cube.x" into the cubeMesh variable. You should now be able to see and control and arrow when you run your program.
h. Add code to rotate the arrow left and right when the keys 'Z' and 'X' are pressed (use the 'RotateY' method).
i. Run the program and rotate the arrow right about 45°. Now move it left and right with 'A' and 'D'.
j. Rotate it some more and try again. What do you notice about the direction that it moves?
k. What you should notice is that the arrow always moves left and right in the same direction: in the X direction of the large grid. This is called the 'world' X direction. Look

closely at the arrow model, and you can see it has its own grid, and it has its own X direction. This is called the arrow's '**local'** X direction. Sometimes we want to move in the local directions; we can do this with these methods:

```
// Move in the local X direction by the given amount (e.g. arrow->MoveLocalX(0.1);)
void MoveLocalX(const float);
// Move in the local Y direction by the given amount
void MoveLocalY(const float);
// Move in the local Z direction by the given amount
void MoveLocalZ(const float);
```

l. Change all the 'Move' methods from the previous exercises to '**MoveLocal'** methods. Now experiment with rotation and movement together. Can you see the difference?

m. Save your project, make sure it is stored where you can access it as you will need it in the next lab.

6. Working with Variables

a. Create a new TL-Engine project named **Lab1C_TestingVariables_Project**. If you have completed the Advanced Movement exercise in the previous Lab worksheet, you can reuse your code in completing the tasks below.

b. Load the cube mesh and then create a cube model.

c. Declare a variable of type float in the initialization section (before the game loop) and name it **cubeSpeed**. Assign an appropriate value to **cubeSpeed**.

d. Move the cube to the right using the variable you just created. Use the code below:

```
cube->MoveX( cubeSpeed );
```

e. You can obtain the current location of a model. For instance, you can obtain the current x coordinate of the cube by using:

```
xCoordinate = cube->GetX();
```

Note that I have used a new variable to store the location of the cube. This new variable is called xCoordinate. The GetX() function returns to the value. In other words, it passes a value back, and this value can either be used directly or assigned to a variable.

f. Now get the cube to stop moving when it reaches 40 along the x-axis, i.e., when GetX( ) returns a value of 40. **Hint:** You will have to declare a variable to store the value returned by GetX(), and you will have to use the if statement to check it.

g. Replace the cube displayed in your program with a sphere.

h. Then update your code so that the sphere bounces from left to right between x = -40 and x = 40. Use a variable to determine the direction of movement.

i. Can you make the sphere roll while it bounces?

Pause and Reverse

a. Expand your project so you can use the keyboard in the following way:

- Whenever the ‘R’ key is pressed, the sphere changes direction.
- The ‘P’ key acts as a pause button for your “game”.

Hints:

- You will need a variable to implement the pause button.
- You will need to use selection to check the state of your variable

7. Basic Square Path

a. Create a new TL-Engine project called **Lab1C_SquarePath_Project**.

b. Load a floor mesh using "**floor.x**", then create a floor model at the **origin** (0, 0, 0).

c. Load a **sphere mesh**, then create a sphere model at position (-75, 10, 75), then follow the instructions below to make it move continuously along a square path.

d. Update your code so that on starting the program, a check is performed to confirm whether the z-position of the sphere is greater than or equal to 75; if true, then the sphere is moved using the **MoveX()** method until its x-position is greater than or equal to 75. Note that you may have to use a nested if to implement this step and the three that follow.

e. Update your code so that if the sphere's x-position is greater than or equal to 75, the sphere is moved along the z-axis using the **MoveZ()** method until it's z-position is less than or equal to -75.

f. Update your code so that if the sphere's z-position is less than or equal to -75, the sphere is moved along the x-axis using the **MoveX()** method until its x-position is less than or equal to -75.

g. Update your code so that if the sphere's x-position is less than or equal to -75, the sphere is moved along the z-axis using the **MoveZ()** method until its z-position is greater than or equal to 75.

h. The sphere should now continuously move in a square path.

i. Can you make the sphere roll in the right direction while it moves? If you find this too complex, move to the next section and come back to this later.